PENSE-BÊTE · TECHNIQUE ET COMMANDES

Gestion des périphériques et architecture E/S

06-550 PSE 25-27 4-20 Windows Server - Gérer les périphériques — source : fiche de révision du module (gemma4, 03/11/2026)

À RETENIR

1. L'I/O Manager centralise les opérations en créant des IRP pour communiquer avec les pilotes.
2. La couche HAL assure l'indépendance du noyau vis-à-vis du matériel et la gestion des interruptions.
3. Le modèle UMDF améliore la stabilité du système car un crash de pilote en mode utilisateur ne provoque pas d'arrêt complet.

Architecture de gestion des E/S

Notion	À retenir
Principes généraux et objectifs	Architecture en couches séparant applications, noyau et matériel. Échanges via **IRP** (I/O Request Packets). Chaîne : Application → API → **I/O Manager** → Pilotes → Matériel.
HAL (Hardware Abstraction Layer)	Couche intermédiaire entre le noyau et le matériel. Assure l'**abstraction**, la gestion des **interruptions** et l'indépendance du noyau vis-à-vis du constructeur.

Composants E/S et Pilotes

Notion	À retenir
Les cinq composants du système E/S Windows	I/O Manager (central, crée les **IRP**), **IRP** (structure de requête), Cache Manager (optimise accès disque), Memory Manager (gère buffers/memory-mapped I/O), Security Reference Monitor (vérifie droits/accès).
Types de pilotes	**Fonction** (gère le périphérique), **filtre** (intercepte les requêtes IRP et ajoute des fonctions), **bus** (gère les contrôleurs).

Organisation et modèles de pilotes

- **Driver Stack (pile)** : Application → Driver filtre (optionnel) → Driver fonction → Driver bus → Matériel.
- **WDM** : ancien modèle bas niveau, puissant mais complexe et peu sécurisé.
- **KMDF** : framework moderne noyau (simplifie mémoire, Plug and Play, énergie, IRP).
- **UMDF** : exécution en mode utilisateur pour une meilleure stabilité (système stable si crash).

Résumé des concepts E/S et périphériques

Notion	À retenir
Interruptions et DMA	Interruptions (**IRQ** classiques, **MSI**, **MSI-X** : meilleure répartition entre processeurs) gérées par **HAL**, noyau et pilotes. **DMA** : accès direct mémoire sans solliciter le CPU (**SSD NVMe**, **RAID**, réseau).
Modes E/S et cycle	Synchrones (**attente**) vs Asynchrones (**Overlapped I/O**, **IOCP**). Cycle : 1. Demande application → 2. **I/O Manager** crée **IRP** → 3. Pile de pilotes → 4. Pilote matériel → 5. Résultat.
Gestion des périphériques	Plug and Play : détection et ressources. États : **Détecté**, **Démarré**, **Suspendu**, **Arrêté**, **Supprimé**. Sécurité : **signature obligatoire**, **Secure boot**, **HVCI**, isolation mémoire.

Administration et outils

Commande	À quoi ça sert
`devmgmt.msc`	Gestionnaire de périphériques (installation/désinstallation pilotes, mise à jour, diagnostic, activation/désactivation)
`verifier /standard /all`	**Driver Verifier** sur tous les pilotes non Microsoft (test uniquement, jamais en production)

Pièges et confusions

- ⚠ Croire qu'une application peut accéder directement au matériel : cela déclenche une exception de protection mémoire.
- ⚠ Croire qu'un pilote UMDF qui plante provoque un écran bleu : le système reste généralement stable.
- ⚠ Activer Driver Verifier en production : réservé aux tests, il peut rendre le système instable.
- ⚠ Croire les conflits de ressources fréquents : ils sont rares grâce au PCI Express et à MSI-X.
- ⚠ Négliger les E/S asynchrones sur un serveur : sans elles, les threads se bloquent et la capacité chute.

Sigles et définitions

HAL	Couche intermédiaire entre le noyau Windows et le matériel physique.
I/O Manager	Composant central du noyau responsable de la gestion de toutes les opérations E/S.
IRP	Structure de données représentant une requête E/S.
KMDF	Framework moderne de développement de pilotes en mode noyau.
UMDF	Framework permettant d'exécuter certains pilotes en mode utilisateur.
IOCP	Mécanisme d'E/S asynchrones utilisé par les applications serveur.
Driver Verifier	Outil permettant de tester les pilotes et détecter les erreurs noyau.

Auto-test

1. Quel composant crée les IRP ?
2. Quel est le rôle du pilote de filtre ?
3. Quelle commande permet d'ouvrir le Gestionnaire de périphériques ?
4. Quel mécanisme permet la détection automatique des périphériques ?
5. Quel type d'interruption permet une meilleure répartition entre les processeurs ?

Réponses : **1.** I/O Manager · **2.** Intercepter les requêtes IRP et ajouter des fonctionnalités supplémentaires. · **3.** devmgmt.msc · **4.** Plug and Play · **5.** MSI-X

À compléter par la fiche, le podcast et le quiz du module. Rédigé à partir de la fiche de révision ; seuils et chiffres à la date de la source.